

ELI — Background Tasks

ELI v2.4.70

September 2026

Contents

ELI Background Tasks & Unified Code Generation	1
1. Unified code generation	1
2. Background task manager (eli/runtime/background_tasks.py)	2
Control summary	2
Scope / honesty	2
Updated for v2.4.70.	

ELI Background Tasks & Unified Code Generation

Two related additions: 1. **All code generation now routes through the verified coding agent** — asking for a script, or self-upgrading, runs the plan → DAG/tree-search → execute → repair → bug-memory pipeline instead of a single LLM shot. 2. **Heavy work runs on background threads** — when a task is estimated heavy (or you ask), ELI starts it on a worker thread, hands back a job id immediately, and you check on it later.

1. Unified code generation

Path	Before	Now
CODE_SOLVE GENERATE_SCRIPT	the coding agent inline single-shot prompt + static checks	the coding agent (unchanged) routes through eli.coding.solve first (plan/DAG/tree- search/verify/repair), saving a verified, top-tier script; falls back to the inline generator only if the agent yields nothing or ELI_GENERATE_SCRIPT_AGENT=0
Self-upgrade (self_improvement.generate_code_patch)	LLM patch + import-verify	now also consults the coding engine’s long-term bug memory : classifies the failure and injects prior fixes for that bug class into the patch prompt

So “write a script”, “generate code”, and “improve ELI” all share one engine and one bug-memory — the same consolidation move applied to recall_memory. Kill switch: ELI_GENERATE_SCRIPT_AGENT=0 restores the legacy inline generator.

2. Background task manager (eli/runtime/background_tasks.py)

In-process, multi-threaded (ThreadPoolExecutor). **Distinct** from eli/planning/jobqueue.py (which runs durable *external subprocess* jobs) — this runs live in-process Python work (agent runs) for the session.

- BackgroundTasks.submit(name, fn, *args) → integer job id; tracks queued → running → done/failed/cancelled, result, error, elapsed, note.
- get(id), list(), cancel(id) (best-effort), wait(id), stats().
- Singleton get_background_tasks(); thread-safe.

When ELI backgrounds a task (eli/coding/cost.py)

should_background(task) decides: - **Explicit phrasing wins**: “in the background / don’t wait / notify me when” → background; “right now / quickly / wait for it” → foreground. - **Otherwise heuristic**: estimate_cost scores heavy-compute signals (simulate/monte-carlo/optimise/benchmark/train/dijkstra/parallel...), multi-component structure, length, and planned-step count. heavy (≥ 0.6) = background.

_maybe_background_codegen (executor) applies this at the top of CODE_SOLVE and GENERATE_SCRIPT: if heavy/explicit, it submits the same action as a background job (re-dispatched with _no_background so the worker doesn’t re-background) and replies “started as job #N — say ‘check job N’.” Kill switch: ELI_CODEGEN_BACKGROUND=0.

Inspecting jobs

- Actions **CHECK_JOB** ({job_id} or parsed from “check job 3”) and **BACKGROUND_JOBS** (list), both in SUPPORTED_ACTIONS.
- Router triggers: “check job N” / “job N”, “background jobs” / “list jobs”.

Control summary

Knob	Default	Effect
ELI_GENERATE_SCRIPT_AGENT	on	GENERATE_SCRIPT uses the coding agent (off ⇒ legacy inline)
ELI_CODEGEN_BACKGROUND	on	auto-background heavy codegen (off ⇒ always foreground)
ELI_AGENT_DAG / ELI_CODING_DAG	on	DAG dispatch / subtask DAG (see dag.md)
ELI_CODING_SANDBOX / *_RUN_TIMEOUT / *_BEAM / *_MAX_ITERS	—	coding-agent execution knobs

Scope / honesty

- **Tested deterministically** (tests/test_background.py): manager run/fail reporting, cost light-vs-heavy + explicit phrasing, and the CHECK_JOB / BACKGROUND_JOBS executor round-trip. No model needed.
- **Threads, not processes**: background tasks share the process; a running thread can’t be force-killed (cancel is best-effort / cooperative). True OS isolation for a job uses the durable subprocess jobqueue: run eli-jobs worker and queue with eli-jobs submit --cmd “...” (jobs survive a restart; one a dead worker left running is marked failed on the next worker start).
- **“LLM deems longer wait”**: currently a deterministic estimator (+ explicit phrasing). The planner’s decomposition (plan_steps) can be fed in for an LLM-informed estimate — hook present, not yet auto-wired.

- **GUI:** the Tasks tab (`eli/gui/tabs/tasks_tab.py`) is the single home for background work: coding jobs and scheduled or overnight tasks (add, edit, cancel). Jobs are also reachable in chat (“check job N”, “background jobs”) and through the action API.